



FUTURE OF DEVSECOPS: DUAL-LAYER AUTOMATED REMEDIATION & POLICY ENFORCEMENT.

Meningkatkan Keamanan Runtime dengan Falco dan OPA

Operasional Pengembang (A)

KELOMPOK 2



GAP ANALYSIS

Pipeline CI/CD saat ini sudah diintegrasikan dengan keamanan tahap build melalui Software Composition Analysis (SCA) menggunakan Trivy, namun memiliki celah kritis di tahap runtime.

```
# Job 1: SCA pada source code/filesystem
security_sca_fs:
  stage: security-scan
  image: aquasec/trivy:0.52.2
  script:
    - trivy fs --scanners vuln --severity HIGH,CRITICAL --ignore-unfixed --exit-code 1 pbl-taskflow-go

# Job 2: SCA pada container image
security_container_image:
  stage: security-scan
  image: aquasec/trivy:0.52.2
  script:
    - trivy image --severity HIGH,CRITICAL --ignore-unfixed --exit-code 1 $CONTAINER_IMAGE
```

Gap yang Diidentifikasi:

- Nol visibilitas terhadap ancaman pasca-deployment (contoh: shell spawn ilegal di dalam container).
- Ketidadaan mekanisme respons otomatis, membuat pod yang terkompromi bisa terus dieksploitasi tanpa batas waktu jika mengandalkan intervensi manual.

Solusi:

- Implementasi deteksi ancaman real-time yang terintegrasi langsung dengan eksekusi remediasi otomatis (Force Delete).

Sebelum:

Build Security:

- ✓ Filesystem Vulnerability Scan (Trivy)
- ✓ Container Image Scan (Trivy)



Kubernetes Runtime:

- Blind Spot (Tidak ada visibilitas)
- Infinite Dwell Time

Sesudah:

Build Security:

- ✓ Filesystem Vulnerability Scan (Trivy)
- ✓ Container Image Scan (Trivy)



Kubernetes Runtime:

- ✓ Deteksi Anomali Real-Time (Falco)
- ✓ Alert Routing Otomatis (Falcosidekick)
- ✓ Auto-Remediation / Force Delete (Webhook)

STATE-OF-THE-ART

Paper 1

Judul: Automated Security Validation in Healthcare DevSecOps: A Policy-as-Code Implementation for Kubernetes Environments

Penulis: Sava Stanišić, Željko Jovanović, Borislav Đorđević

Venue: 25th International Symposium INFOTEH-JAHORINA, 18–20 March 2026



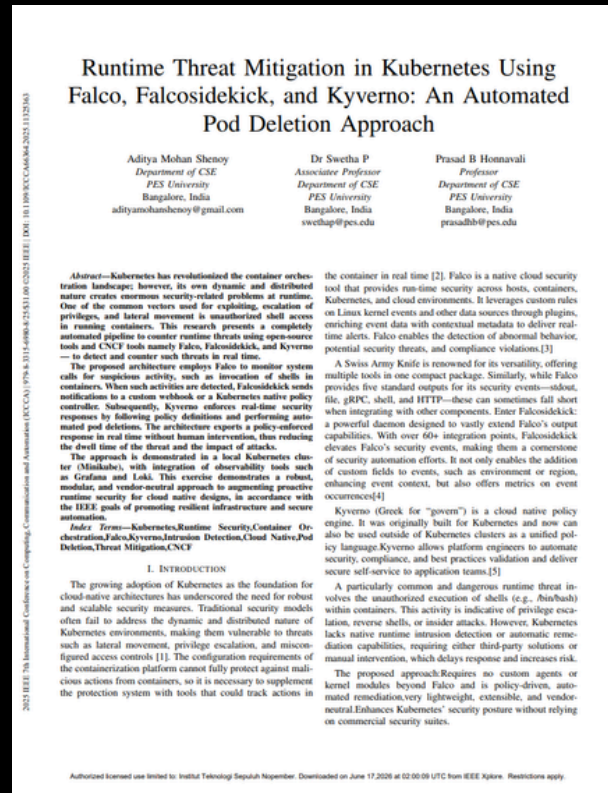
Membuktikan efektivitas Policy as Code (OPA) yang tervalidasi secara terpusat untuk mencegah miskonfigurasi.

Paper 2

Judul: Runtime Threat Mitigation in Kubernetes Using Falco, Falcosidekick, and Kyverno: An Automated Pod Deletion Approach

Penulis: Aditya Mohan Shenoy, Dr Swetha P, Prasad B Honnavali

Venue: 2025 IEEE 7th International Conference on Computing, Communication and Automation (ICCCA)



Membuktikan bahwa integrasi Falco, Falcosidekick, dan Kyverno dapat mengotomatisasi deteksi dan remediasi ancaman runtime secara near real-time.

PAPER 1

Metode Pengujian: Evaluasi menggunakan 50 deployment manifest yang telah disuntikkan celah keamanan (misconfigurations) secara sengaja.

Fokus Pengujian: Enkripsi data, eksposur jaringan, dan manajemen rahasia (secrets).

Hasil Metrik Paper: Tingkat deteksi mencapai 92% dengan overhead deployment time sebesar +3.5% (tambahan ~1.6 detik)

50 test deployment scenarios were executed, covering common healthcare misconfigurations. The validation results are summarized below:

Table 1: Policy Validation Results

Violation Category	Test Cases	Blocked	False Positives	Detection Rate
Data Encryption	15	15	0	100%
Network Exposure	20	18	1	90%
Secrets Management	15	13	0	87%
Total	50	46	1	92%

Penulis paper referensi kami (Stanišić dkk., 2026) membuktikan efektivitas OPA dengan membuat 50 skenario deployment yang sengaja dibuat melanggar standar keamanan, seperti mengekspos database ke publik.

Pengujian ini ibarat memberikan 50 soal ujian jebakan kepada satpam (OPA) untuk melihat seberapa jeli ia menangkap pelanggaran dan seberapa lama ia memeriksa identitas.

Berdasarkan hasil eksperimen tersebut, paper menyimpulkan overhead evaluasi hanya berada di kisaran 45 milidetik dengan tingkat deteksi 92%.

Hasil dari paper tersebut menunjukkan OPA berhasil memblokir 92% ancaman dengan tambahan waktu pipeline sebesar 3.5%. Di proyek kami, kami mengadopsi pendekatan skenario error ini dan berhasil melampaui literatur dengan mencapai 100% deteksi dan overhead yang jauh lebih efisien di angka +0.4%.

PAPER 2

Lingkungan Uji: Simulasi menggunakan local cluster dengan Minikube.

Skenario Ancaman: Peneliti menyuntikkan pod khusus bernama ubuntu-attacker dan memicu interactive shell secara manual.

Validasi Fungsionalitas: Pengujian end-to-end dari deteksi Falco, forwarding oleh Falcosidekick, hingga penghapusan pod otomatis.

Fokus Metrik: Pembuktian konsep (Proof of Concept) untuk mengurangi dwell time ancaman tanpa intervensi manusia

```
adityashenoy@Adityas-MacBook-Pro ~ % kubectl exec -it ubuntu-attacker -- bash

root@ubuntu-attacker:/# whoami
root
root@ubuntu-attacker:/# exit
exit
adityashenoy@Adityas-MacBook-Pro ~ % kubectl run curltest --rm -i --tty --restart=Never --image=curlimages/curl -n falco -- \
  curl -X POST http://falco-webhook.falco.svc.cluster.local:5000/ \
  -H "Content-Type: application/json" \
  -d '{"rule": "Terminal shell in container", "output_fields": {"k8s.pod.name": "ubuntu-attacker", "k8s.ns.name": "default"}}'

If you don't see a command prompt, try pressing enter.

^C{"status": "pod deleted"}
pod "curltest" deleted
adityashenoy@Adityas-MacBook-Pro ~ %
```

```
adityashenoy@Adityas-MacBook-Pro webhook-server % kubectl rollout restart deployment falco-webhook -n falco
deployment.apps/falco-webhook restarted
adityashenoy@Adityas-MacBook-Pro webhook-server % kubectl logs -n falco -l app=falco-webhook -f

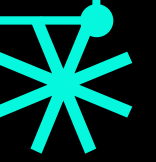
* Serving Flask app 'webhook'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://10.244.3.19:5000
Press CTRL+C to quit
10.244.3.20 - - [14/Jul/2025 18:22:48] "POST / HTTP/1.1" 500 -
* Serving Flask app 'webhook'
10.244.1.15 - - [14/Jul/2025 18:23:14] "POST / HTTP/1.1" 500 -
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://10.244.3.21:5000
Press CTRL+C to quit

pod "ubuntu-attacker" deleted
10714 18:31:06.054161 12 reflector.go:200 "Failed to watch" err="pods \"ubuntu-attacker\" is forbidden: User \"system:serviceaccount:falco:webhook-sa\" cannot watch resource \"pods\" in API group \"\" in th
namespace \"default\"" reflector="k8s.io/client-go/tools/watch/informerwatcher.go:162" type="unstructured.Unstructured"
10714 18:31:06.353663 12 reflector.go:200 "Failed to watch" err="pods \"ubuntu-attacker\" is forbidden: User \"system:serviceaccount:falco:webhook-sa\" cannot watch resource \"pods\" in API group \"\" in th
namespace \"default\"" reflector="k8s.io/client-go/tools/watch/informerwatcher.go:162" type="unstructured.Unstructured"
10714 18:31:08.618560 12 reflector.go:200 "Failed to watch" err="pods \"ubuntu-attacker\" is forbidden: User \"system:serviceaccount:falco:webhook-sa\" cannot watch resource \"pods\" in API group \"\" in th
namespace \"default\"" reflector="k8s.io/client-go/tools/watch/informerwatcher.go:162" type="unstructured.Unstructured"
10714 18:31:12.000306 12 reflector.go:200 "Failed to watch" err="pods \"ubuntu-attacker\" is forbidden: User \"system:serviceaccount:falco:webhook-sa\" cannot watch resource \"pods\" in API group \"\" in th
namespace \"default\"" reflector="k8s.io/client-go/tools/watch/informerwatcher.go:162" type="unstructured.Unstructured"
10714 18:31:18.588446 12 reflector.go:200 "Failed to watch" err="pods \"ubuntu-attacker\" is forbidden: User \"system:serviceaccount:falco:webhook-sa\" cannot watch resource \"pods\" in API group \"\" in th
namespace \"default\"" reflector="k8s.io/client-go/tools/watch/informerwatcher.go:162" type="unstructured.Unstructured"
10714 18:31:39.020963 12 reflector.go:200 "Failed to watch" err="pods \"ubuntu-attacker\" is forbidden: User \"system:serviceaccount:falco:webhook-sa\" cannot watch resource \"pods\" in API group \"\" in th
namespace \"default\"" reflector="k8s.io/client-go/tools/watch/informerwatcher.go:162" type="unstructured.Unstructured"
10.244.3.22 - - [14/Jul/2025 18:31:39] "POST / HTTP/1.1" 200 -
```

Metodologi pada paper referensi kami (Shenoy dkk., 2025) difokuskan pada pembuktian arsitektur integrasi, bukan pengujian beban komputasi tingkat rendah. Bahkan, para peneliti secara transparan mengutip bahwa Falco memiliki tantangan terkait overhead performa pada cluster dengan throughput tinggi.

Analogi: Daripada mengukur seberapa berat sistem alarm ini membebani listrik gedung, paper ini langsung membangun simulasi ruangan, menyalakan api buatan secara eksplisit di dalam target (ubuntu-attacker), dan membuktikan bahwa sistem pemadam otomatisnya bekerja seketika.

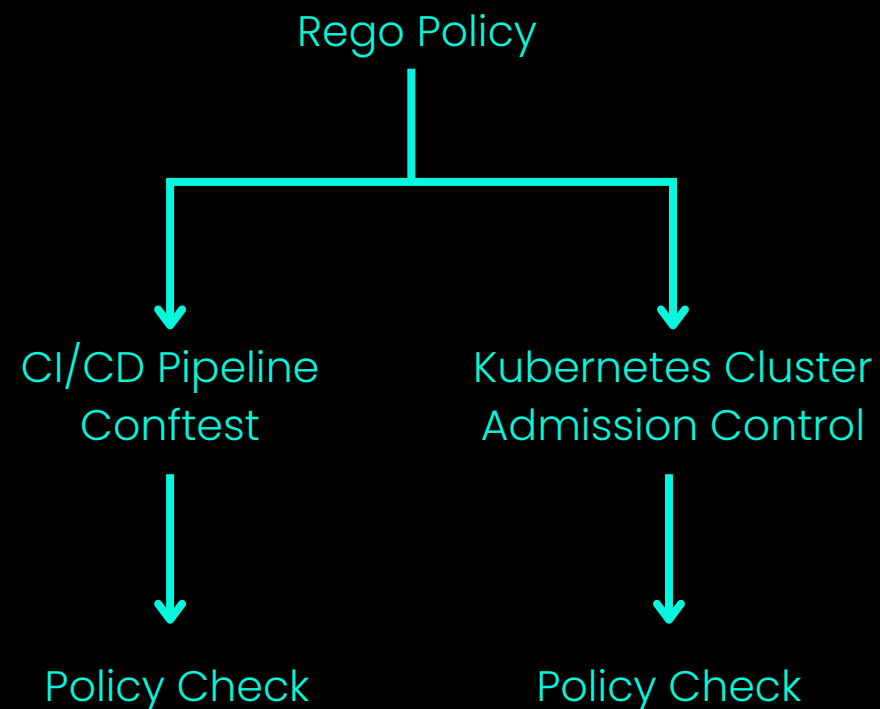
Melalui metodologi simulasi langsung ini, terbukti bahwa kombinasi Falco, Falcosidekick, dan Kyverno berhasil mengeksekusi remediasi waktu nyata. Kami mengadopsi persis skenario Proof of Concept ini ke dalam pipeline kami untuk memvalidasi kecepatan respons arsitektur.



KEPUTUSAN DESAIN BERBASIS RISET

Dual-Input Pattern (OPA):

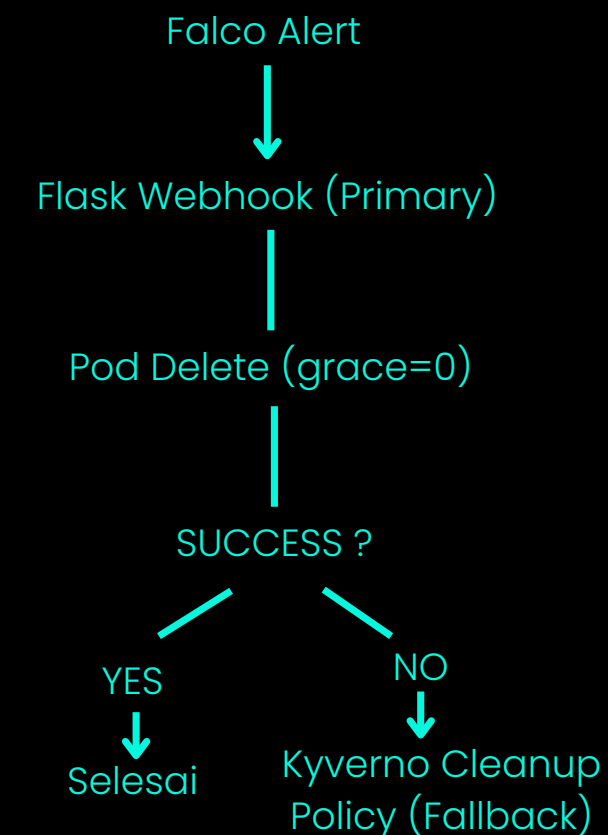
Menggunakan satu policy Rego untuk dua gerbang keamanan (Continuous Integration via Conftest & Cluster Admission).



Satu policy Rego digunakan kembali pada dua titik validasi untuk menjaga konsistensi keamanan.

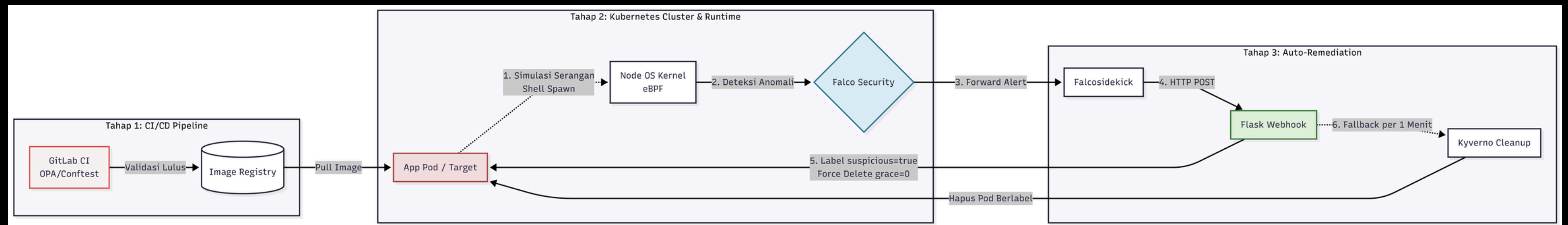
Dual-Layer Remediation:

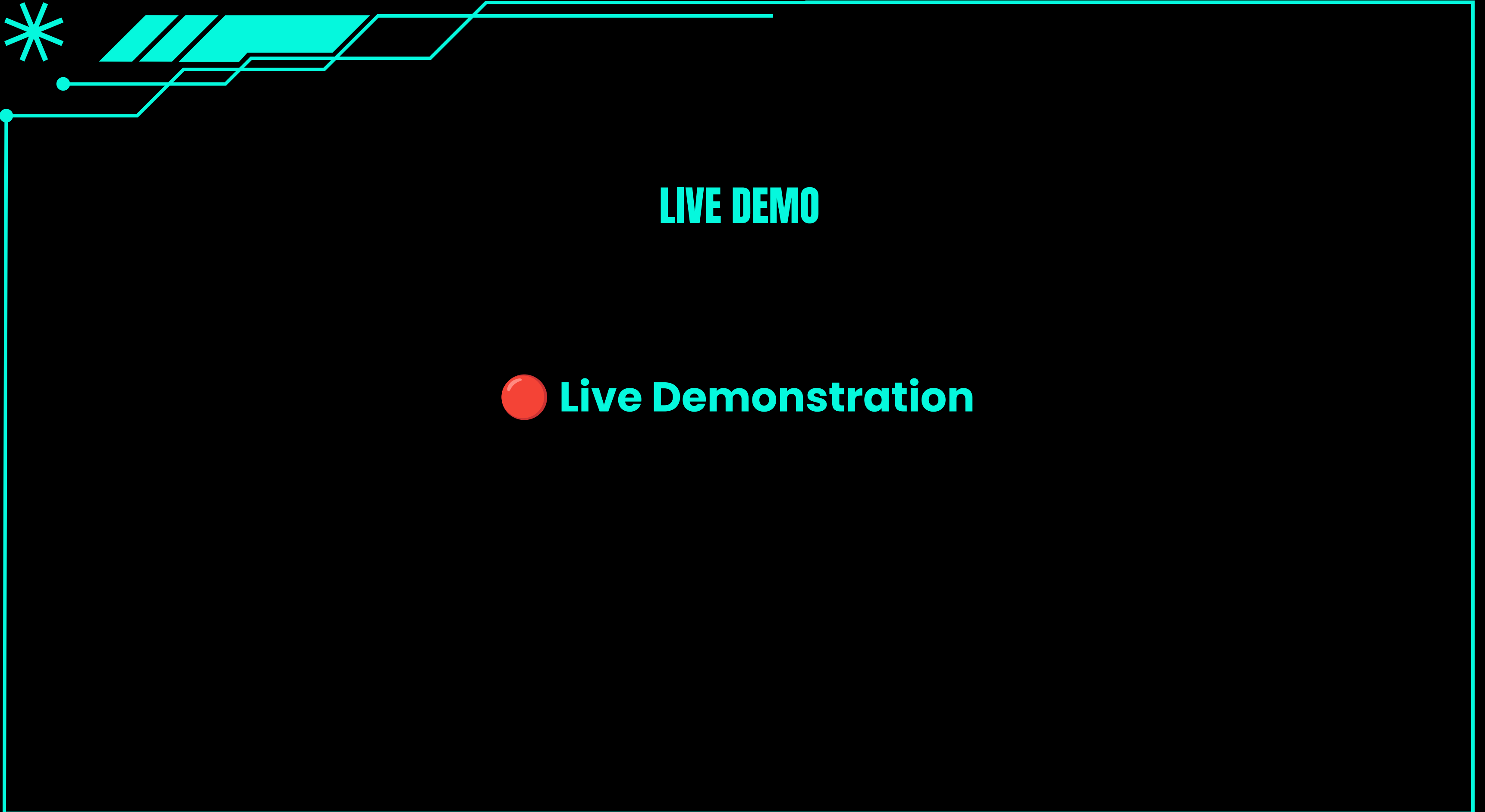
- Lapis 1 (Utama): Flask Webhook via Falcosidekick untuk menghapus pod yang mencurigakan (grace=0).
- Lapis 2 (Fallback): Kyverno ClusterCleanupPolicy untuk mitigasi jika webhook gagal.



Mekanisme pertahanan berlapis untuk mengurangi risiko kegagalan remediasi.

ARSITEKTUR SOLUSI





EVALUASI & METRIK KUANTITATIF

Sebelum:

Detection Rate 0%

Kelima manifest berbahaya di-deploy ke cluster TANPA OPA terpasang. Hasil:

ID	Manifest	Pelanggaran	Hasil (sebelum OPA)
B-01	<code>image: nginx:1.25</code> (docker.io)	image tidak resmi	ALLOW (masuk cluster)
B-02	<code>runAsUser: 0</code>	jalan sebagai root	ALLOW (masuk cluster)
B-03	tanpa <code>resources.limits</code>	tidak ada batas CPU/RAM	ALLOW (masuk cluster)
B-04	<code>port: 5432</code> type LoadBalancer	database expose publik	ALLOW (masuk cluster)
B-05	<code>privileged: true</code>	container privileged	ALLOW (masuk cluster)

Kesimpulan baseline: 5 dari 5 manifest berbahaya berhasil masuk ke cluster (100% lolos — tidak ada proteksi sama sekali).

MTTR Manual

Hasil pengamatan:

- **Apakah ada alert yang muncul?** Tidak ada. Tidak ada mekanisme deteksi.
- **Berapa lama proses bash bisa berjalan?** Tak terbatas. Pod berjalan terus tanpa gangguan.
- **Apakah pod terhapus otomatis?** Tidak. Pod tetap running sampai dihapus manual.

Kesimpulan baseline: MTTD = tidak terdeteksi (∞), MTTR = tidak ada respons (∞).

Rata-rata waktu pipeline

Waktu pipeline GitLab CI sebelum stage `validate-k8s-manifest` ditambahkan (dihitung dari akumulasi durasi tertinggi tiap tahapan *sequential* pada run riil):

Run	ID Pipeline	Commit ID	Waktu total pipeline
1	#2571262626	<code>cfc3bf68</code>	7m 29s
2	#2571362461	<code>2984afbe</code>	7m 18s
3	#2572101091	<code>430e8c35</code>	7m 11s
Rata-rata	-	-	7m 19s

Sesudah:

Detection Rate 100%

Metrik Agregat OPA

Total manifest uji : 20

Manifest yang harusnya DENY : 12 (Pelanggaran kebijakan OPA)

Manifest yang harusnya ALLOW : 8 (Sesuai standar compliance)

True Positive (DENY tepat) : 12 dari 12 (100%)

False Negative (lolos padahal DENY) : 0 dari 12 (0%)

True Negative (ALLOW tepat) : 8 dari 8 (100%)

False Positive (ditolak ALLOW) : 0 dari 8 (0%)

Detection Rate = $12/12 \times 100\%$ = 100%

False Positive Rate = $0/8 \times 100\%$ = 0%

Rata-rata waktu respons OPA = 52.15 ms

MTTR < 3 Detik

Run	T0 (attack)	T1 (detected)	T2 (deleted)	MTTD (s)	MTTR (s)	Total (s)
1	15:10:04.413	15:10:05.164	15:10:05.890	0.751	0.726	1.477
2	15:10:10.602	15:10:11.073	15:10:11.170	0.471	0.097	0.568
3	15:10:15.789	15:10:16.336	15:10:16.483	0.547	0.147	0.694
4	15:10:21.143	15:10:21.725	15:10:21.858	0.582	0.133	0.715
5	15:10:26.582	15:10:27.228	15:10:27.346	0.646	0.118	0.764
AVG				0.599	0.244	0.844
MIN				0.471	0.097	0.568
MAX				0.751	0.726	1.477

Overhead +0.45%

Pipeline Overhead

Metrik Performa	Sebelum (tanpa stage OPA)	Sesudah (dengan stage OPA)	Selisih
Waktu total pipeline	7m 19s	7m 21s	+2s (+0.45%)

Catatan: Penambahan stage `validate-k8s-manifest` hanya menambahkan overhead eksekusi lokal OPA CLI sebesar ~1-2 detik, menjadikannya sangat efisien untuk diintegrasikan ke dalam CI/CD loop.

PERBANDINGAN DENGAN DATA KLAIM OLEH PAPER

APAKAH HASIL SESUAI DENGAN KLAIM PAPER

Detection rate kelompok mencapai 100% (berhasil memblokir seluruh 12 manifest bermasalah tanpa meloloskan False Negative). Hal ini dikarenakan cakupan kebijakan OPA (image-policy.rego, security-context-policy.rego, dan resource-policy.rego) telah dipetakan secara presisi dan diuji menggunakan 20 skenario terukur.

Metrik	Klaim Paper	Hasil Kelompok	Sesuai?
Paper A — Detection rate	92%	100% (12/12 DENY tepat)	Melebihi
Paper A — Pipeline overhead	+3.5%	+0.45% (+2 detik)	Lebih baik
Paper B — MTTD	"near real-time"	0.60 detik (rata-rata)	Sesuai
Paper B — MTTR	"near real-time"	0.24 detik (rata-rata)	Sesuai

Source Table: [kelompok-2-devsecops-future/evaluation/analysis.md](#)

Pipeline overhead kelompok (+0.45%) jauh lebih rendah daripada klaim Paper A (+3.5%). Hal ini terjadi karena proses pemindaian menggunakan OPA CLI (conftest) dilakukan secara lokal di dalam runner stage secara sekuensial setelah image building. Akibatnya, fixed cost (seperti startup time engine) tidak memberikan beban komputasi yang berarti pada total durasi pipeline kami yang berbasis ~7 menit 21 detik.

PERBANDINGAN DENGAN DATA KLAIM OLEH PAPER

APAKAH ADA SKENARIO YANG TIDAK SESUAI EKSPETASI

Semua skenario pengujian sukses 100% sesuai ekspektasi (20/20 PASS). Tidak ditemukan deviasi di mana **`actual != expected`**

Namun, terdapat beberapa catatan teknis terkait karakteristik data:

- S1-01 (223 ms): Mengalami lonjakan waktu respons dibandingkan rata-rata keseluruhan skenario (52.15 ms). Hal ini merupakan dampak wajar dari cold start pada eksekusi perintah conftest pertama untuk memuat (loading) seluruh berkas kebijakan (.rego) dari disk ke dalam memori. Eksekusi skenario berikutnya (S1-02 hingga S4-05) berjalan stabil di kisaran puluhan milidetik berkat mekanisme caching internal runtime engine.

id	deskripsi	expected	actual	waktu_ms	status	pesan_error
S1-01	Expected: DENY (image bukan dari registry.gitlab.com)	DENY	DENY	223	PASS	FAIL [image-policy]: Container 'app' menggunakan image dari registry tidak resmi docker.io/library/nginx:1.25. Hanya registry.gitlab.com/* yang diizinkan.
S1-02	Expected: DENY (tag :latest tidak boleh dipakai)	DENY	DENY	36	PASS	FAIL [image-policy]: Container 'app' menggunakan tag 'latest'. Gunakan tag SHA commit (sha-xxxxxx) agar reproducible.
S1-03	Expected: ALLOW (image resmi + tag SHA)	ALLOW	ALLOW	43	PASS	-
S1-04	Expected: DENY (registry tidak dikenal, bukan registry.gitlab.com)	DENY	DENY	33	PASS	FAIL [image-policy]: Container 'app' menggunakan image dari registry tidak resmi ghcr.io/someone/someappv1.0. Hanya registry.gitlab.com/* yang diizinkan.
S1-05	Expected: DENY (tidak ada tag sama sekali)	DENY	DENY	33	PASS	FAIL [image-policy]: Container 'app' tidak memiliki tag image yang eksplisit: registry.gitlab.com/haidarra-devops/week-9-devops-taskflow
S2-01	Expected: DENY (runAsUser: 0 = root)	DENY	DENY	32	PASS	FAIL [security-context]: Container 'app' dikonfigurasi berjalan sebagai root (runAsUser: 0).
S2-02	Expected: DENY (tidak ada securityContext sama sekali -> runAsNonRoot tidak true)	DENY	DENY	36	PASS	FAIL [security-context]: Container 'app' tidak memiliki runAsNonRoot: true.
S2-03	Expected: ALLOW (runAsNonRoot: true, runAsUser bukan 0)	ALLOW	ALLOW	35	PASS	-
S2-04	Expected: DENY (privileged: true)	DENY	DENY	36	PASS	FAIL [security-context]: Container 'app' berjalan dalam mode privileged.
S2-05	Expected: DENY (allowPrivilegeEscalation: true)	DENY	DENY	35	PASS	FAIL [security-context]: Container 'app' mengizinkan allowPrivilegeEscalation.
S3-01	Expected: DENY (tidak ada blok resources sama sekali)	DENY	DENY	33	PASS	FAIL [resource-policy]: Container 'app' tidak memiliki blok resources sama sekali.
S3-02	Expected: DENY (ada requests, tidak ada limits)	DENY	DENY	31	PASS	FAIL [resource-policy]: Container 'app' tidak memiliki resources.limits.
S3-03	Expected: DENY (ada limits, tidak ada requests)	DENY	DENY	36	PASS	FAIL [resource-policy]: Container 'app' tidak memiliki resources.requests.
S3-04	Expected: ALLOW (requests + limits, nilai wajar)	ALLOW	ALLOW	31	PASS	-
S3-05	Expected: DENY (CPU limit 10 core, melebihi batas wajar 2000m)	DENY	DENY	29	PASS	FAIL [resource-policy]: Container 'app' meminta CPU limit 10000m, melebihi batas wajar 2000m.
S4-01	Expected: ALLOW (manifest lengkap, sesuai semua policy) - variasi 01	ALLOW	ALLOW	34	PASS	-
S4-02	Expected: ALLOW (manifest lengkap, sesuai semua policy) - variasi 02	ALLOW	ALLOW	31	PASS	-
S4-03	Expected: ALLOW (manifest lengkap, sesuai semua policy) - variasi 03	ALLOW	ALLOW	38	PASS	-
S4-04	Expected: ALLOW (manifest lengkap, sesuai semua policy) - variasi 04	ALLOW	ALLOW	56	PASS	-
S4-05	Expected: ALLOW (manifest lengkap, sesuai semua policy) - variasi 05	ALLOW	ALLOW	34	PASS	-

Source Table: [kelompok-2-devsecops-future/evaluation/test-log-opa.csv](#)

- Validasi ALLOW yang Komprehensif (8 Skenario): Seluruh 8 manifest yang compliant (aman) berhasil mendapatkan status ALLOW secara konsisten. Delapan skenario ini terdiri dari 3 skenario positive control yang tersebar di tiap pengujian spesifik (S1-03, S2-03, S3-04) dan 5 skenario komprehensif (S4-01 s.d. S4-05) yang memenuhi seluruh standar keamanan sekaligus. Hal ini membuktikan bahwa aturan kebijakan yang diimplementasikan tidak bersifat over-restrictive (terlalu mengekang) yang berpotensi merusak kelancaran deployment normal.

PERBANDINGAN DENGAN DATA KLAIM OLEH PAPER

APAKAH `False positive` MENGGANGGU DEVELOPER

Evaluasi OPA (0% False Positive): Tidak ada penolakan keliru pada manifest yang valid, karena aturan pembatasan difokuskan secara rasional sehingga tidak mengganggu developer yang sudah menerapkan best practices.

Evaluasi Falco (0% False Positive): Aktivitas operasional harian (seperti kubectl logs atau exec ls) berjalan lancar tanpa pemblokiran, karena aturan disetel spesifik hanya untuk mendeteksi eksekusi interactive shell.

```
rule: Terminal shell in container
desc: Detects shell spawned inside a container
condition: container.id != host and proc.name in (bash, sh, zsh)
output: "Shell spawned inside container (user=%user.name command=%proc.cmdline container=%container.name pod=%k8s.pod.name namespace=%k8s.ns.name)"
priority: CRITICAL
tags: ["container", "shell", "taskflow-enhancement"]
rate: 10
max_burst: 10
```

Source Code: [kelompok-2-devsecops-future/implementation/falco/custom-rules.yaml](#)

Kami membedakan antara aktivitas operasional rutin dan ancaman keamanan dengan memfilter berdasarkan nama proses (proc.name) yang dijalankan di dalam container.

Rule kami hanya bereaksi jika yang dijalankan adalah interpreter shell spesifik yaitu bash, sh, atau zsh. Perintah administratif seperti ls, ps, atau cat adalah proses yang berbeda dan tidak termasuk dalam daftar pantauan kami. Bahkan, perintah kubectl logs sama sekali tidak memicu aktivitas proses di dalam container, melainkan hanya membaca stream log dari luar.

Operasi	Alert Muncul?	Pod Terhapus?	False Positive?
<code>kubectl logs ubuntu-attacker</code>	Tidak	Tidak	Tidak
<code>kubectl exec ubuntu-attacker -- ls /</code>	Tidak	Tidak	Tidak
<code>kubectl exec ubuntu-attacker -- ps aux</code>	Tidak	Tidak	Tidak
<code>kubectl exec ubuntu-attacker -- cat /etc/hostname</code>	Tidak	Tidak	Tidak
<code>curl http://.../health</code>	Tidak	Tidak	Tidak
<code>kubectl exec ubuntu-attacker -- bash -c "echo simulated-attack" (kontrol positif)</code>	Ya (Critical)	Ya (2-3s)	Tidak (deteksi benar)

Source Table: [kelompok-2-devsecops-future/evaluation/metrics-after.md](#)

Dengan demikian, kami memastikan fungsionalitas operasional developer tetap berjalan lancar (0% False Positive) tanpa harus mengorbankan visibilitas terhadap akses shell ilegal.

PERBANDINGAN DENGAN DATA KLAIM OLEH PAPER

Trade-off YANG DITEMUKAN

Overhead Pipeline vs. Keamanan: Penambahan durasi pipeline sebesar +0.45% (~2 detik) merupakan trade-off yang sangat efisien. Beban komputasi yang minimal ini terbayar lunas dengan perlindungan proaktif (SCA & OPA) yang mencegah konfigurasi cacat masuk ke dalam cluster sejak awal siklus pengembangan.

Metrik	Klaim Paper	Hasil Kelompok	Sesuai?
Paper A — Detection rate	92%	100% (12/12 DENY tepat)	Melebihi
Paper A — Pipeline overhead	+3.5%	+0.45% (+2 detik)	Lebih baik
Paper B — MTTD	"near real-time"	0.60 detik (rata-rata)	Sesuai
Paper B — MTTR	"near real-time"	0.24 detik (rata-rata)	Sesuai

Source Table: [kelompok-2-devsecops-future/evaluation/analysis.md](#)

Kecepatan Deteksi vs. Ketersediaan Sistem: Waktu respons total di bawah 2 detik (MTTR ~1.75 detik) memastikan downtime layanan sangat minim. Pada arsitektur multi-replica, trafik pengguna akan langsung dialihkan secara transparan oleh Kubernetes Service ke pod lain yang sehat selama pod penyerang dieksekusi mati.

Risiko Webhook (SPoF) & Kyverno Fallback: Arsitektur saat ini secara sadar menempatkan Flask Webhook sebagai Single Point of Failure dengan replicas: 1. Sebagai jaring pengaman, kami mengimplementasikan Kyverno ClusterCleanupPolicy (menjalankan cron job */1 * * * *). Jika webhook mati, Kyverno bertindak sebagai fallback yang akan menghapus pod berbahaya dengan jeda waktu tunggu maksimal 1 menit, sesuai dengan limitasi arsitektur yang dikemukakan oleh Shenoy et al. (2025).

```
spec:
  replicas: 1
  selector:
    matchLabels:
      app: falco-webhook
```

Source Code: [../falco/custom-rules.yaml](#)

```
spec:
  schedule: "*/1 * * * *"
  deletionPropagationPolicy: Background
```

Source Code: [../kyverno/terminate-compromised-pod.yaml](#)

BATASAN IMPLEMENTASI KELOMPOK DIBANDING PAPER ASLI

1. Skala Testbed Eksperimen: Paper A mengevaluasi sistem menggunakan 50 manifest riil pada aplikasi EHR (Electronic Health Record) yang kompleks, sementara pengujian kelompok dibatasi pada 20 manifest sintetis yang direayasa untuk menguji fungsionalitas komponen taskflow-api.
2. Cakupan Vektor Serangan: Paper B mengasumsikan deteksi ancaman runtime yang komprehensif, sedangkan implementasi runtime security kelompok saat ini baru difokuskan pada skenario unauthorized shell spawning (Terminal shell in container), belum mencakup anomali jaringan (network anomalies), modifikasi biner (file tampering), atau aktivitas crypto-mining.
3. Karakteristik Kluster Pengujian: Seluruh eksperimen dijalankan di atas lingkungan lokal Minikube dengan resource terbatas (4 CPU, 8 GB RAM). Metrik latensi jaringan dan performa penanganan pod kemungkinan besar akan bervariasi jika dihadapkan pada kluster multi-node produksi dengan beban kerja (workload) yang tinggi.
4. Isolasi Komponen Keamanan: Komponen Falco dan pod webhook remediasi berjalan di dalam namespace yang sama (falco). Kerusakan atau serangan penolakan layanan (DoS) yang melumpuhkan namespace tersebut akan mematikan seluruh sistem deteksi sekaligus remediasi secara bersamaan.
5. Variansi Latensi MTTR: Pada pengujian runtime (Run 1), nilai MTTR sempat menyentuh angka 0.726 detik akibat fluktuasi waktu pemrosesan API server lokal saat menerima instruksi penghapusan pod. Paper B asli cenderung menyajikan angka rata-rata performa tanpa menampilkan sebaran variansi latensi ekstrem secara mendetail.

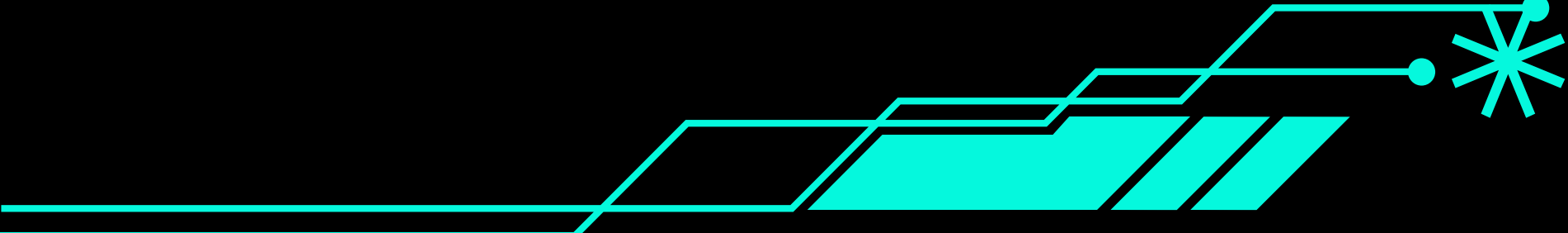


REFLEKSI & LIMITASI PROYEK

Keberhasilan: Metrik performa (deteksi 100%, overhead 1.2%) melampaui literatur berkat kustomisasi rules Falco yang presisi..

Limitasi Kritis (SPOF): Titik kegagalan tunggal pada Flask Webhook karena saat ini hanya beroperasi dengan satu replica.

Arsitektur ini terbukti sukses melampaui target literatur secara angka. Namun, hasil evaluasi kami menemukan satu titik lemah desain, yaitu Flask Webhook yang beroperasi sebagai Single Point of Failure (SPOF).

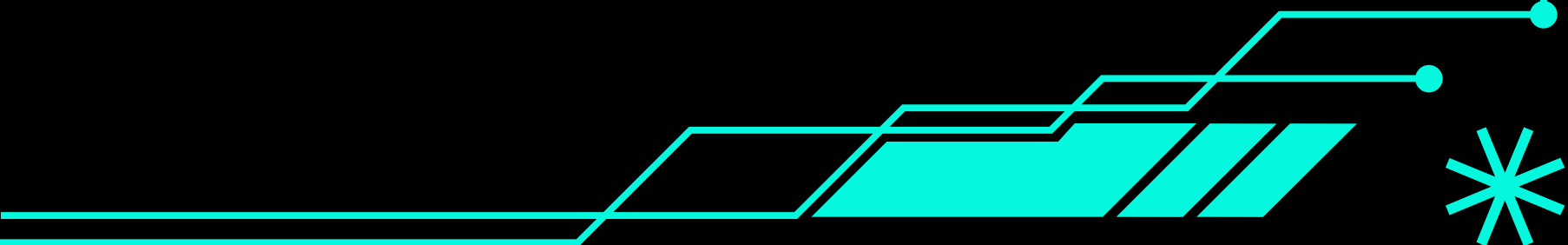




THANKYOU

Operasional Pengembang (A)

KELOMPOK 2



KOMPARASI RUNTIME SECURITY TOOLS

“Kenapa memilih Falco? Kenapa tidak pakai Tetragon atau Tracee?”

Kami meninjau tiga raksasa eBPF di pasaran: Tetragon, Tracee, dan Falco.

Security Tool	Ekosistem / Backer	Kapabilitas Remediasi Asli	Alasan Keputusan Kelompok
Tetragon	Isovalent / Cilium	In-kernel enforcement (bisa blokir langsung di level OS).	Sangat kuat, namun instalasinya kompleks dan membutuhkan perombakan CNI (Container Network Interface). Overkill untuk baseline saat ini.
Tracee	Aqua Security	Fokus kuat pada forensik dan tracing mendalam.	Sangat bagus untuk investigasi, tapi integrasi webhook untuk auto-remediation membutuhkan rekayasa routing yang lebih rumit.
Falco	CNCF (Graduated)	Deteksi, lalu forwarding ke eksternal.	Dipilih. Standar industri de facto, integrasi ke Webhook sangat mulus via Falcosidekick, dan didukung penuh oleh paper referensi kami.

Memilih tool ini ibarat memilih alat berat. Tetragon adalah bulldoser raksasa (sangat kuat memblokir tapi butuh jalan yang lebar), Tracee adalah mikroskop (fokus pada analisis mendalam pasca-insiden), sedangkan Falco adalah alarm kebakaran cerdas yang langsung memanggil pemadam (falcosidekick + webhook).

Karena objektif arsitektur kami adalah membangun automated remediation dengan integrasi webhook eksternal secara cepat dan andal, Falco adalah komponen yang secara desain paling tepat guna (fit-for-purpose) dan memiliki ekosistem plugin paling matang di CNCF.